



South East Technological University

Waterford

School of Science and Computing

Final Year Project

BSc (Hons) in Computer Forensics & Security

System for Deploying Security and Forensics Classroom Lab

Author

Jorja Holland

Supervisor

Jimmy McGibney

January 2026

Contents

Plagiarism Declaration	v
List of Abbreviations	vi
Acknowledgements	vii
1 Introduction & Background	1
1.1 Introduction	1
1.2 Background	1
1.3 Motivation	1
1.4 Problem Statement	1
1.4.1 Industry Example	2
1.5 Project Contributions & Impact	2
2 Aims & Objectives	3
2.0.1 Aim	3
2.0.2 Objectives	3
3 Literature Review	4
4 Architecture & Design	7
4.1 Architecture Model	7
4.2 Design of Website Pages	9
4.2.1 Registration and Confirmation	9
4.2.2 Components of Registration and Confirmation Page	9
4.2.3 Choosing Roles	10
4.2.4 Login	10
4.2.5 Account	10
4.2.6 Labs Documentation	11
4.2.7 Components to Lab Documentation Page	11
4.2.8 Dashboard	12
4.2.9 Launch Templates	12
4.2.10 Components of Creating Launch Templates Page	13
4.2.11 Launching Instances from Templates	13
4.2.12 Components to allow for connection to instances	14
4.3 User Stories	14
4.4 Risks & Mitigations	15
5 Implementation Methodology	16
5.1 Methodology	16
5.1.1 Agile & Scrum	16
5.1.2 Scrum Artifacts	16
5.1.3 Sprints, Sprint Review & Backlog Refinement	16

5.1.4	Burndown Charts	16
5.2	Sprint 1	17
5.3	Sprint 2	17
5.4	Sprint 3	17
5.5	Sprint 4	17
5.6	Sprint 5	18
5.7	Version Control	18
5.8	CI/CD	18
6	Tool & Technologies	19
6.1	AWS	19
6.2	AWS Academy	19
6.3	Apache Guacamole	19
6.4	Overleaf	19
6.5	Git and GitHub	19
6.6	PyCharm	20
6.7	Trello	20
7	Programming Languages & Frameworks	21
7.1	Python	21
7.2	Flask	21
7.3	Boto3	22
7.4	Jinja2	22
7.5	HTML	22
7.6	CSS	22
7.7	JavaScript	22
8	System Testing & Evaluation	24
8.1	Testing Strategy	24
8.2	Testing Approach	24
8.3	Future Testing	24
8.4	Evaluation	25
9	Future Product Developments	26
10	Conclusion	27
A	Design of Website Pages (Figma & Website)	32
A.1	Choose Role	32
A.2	Register	32
A.3	Confirm Account	33
A.4	Login Page	34
A.5	Account	35
A.6	Dashboard	36
A.7	My Instances	37
A.8	Labs Available	38
A.9	Labs Upload	39
A.10	Templates Available	40
A.11	Template Upload	41

B	Sprint Burndown Charts	42
B.1	Reading the Burndown Charts	42

List of Figures

4.1	LearnSec Architecture Diagram	8
A.1	Choose Role website page	32
A.2	Choose Role Figma design	32
A.3	Register website page	32
A.4	Register Figma design	33
A.5	Confirm Account website page	33
A.6	Confirm Account Figma design	34
A.7	Login Page website page : same for Educator and Student	34
A.8	Login Page figma design : same for Educator and Student	35
A.9	Account website page	35
A.10	Account Page Figma design	36
A.11	Dashboard website page	36
A.12	Dashboard Figma design	37
A.13	My instances website page	37
A.14	My Instances Figma design	38
A.15	Labs Available website page for Students	38
A.16	Labs Available website page for Educators	38
A.17	Lab Available Figma design	39
A.18	Lab Upload website page	39
A.19	Lab Upload Figma design	39
A.20	Templates Available website page for Students	40
A.21	Templates Available website page for Educators	40
A.22	Templates Available Figma design	40
A.23	Template Upload website page	41
A.24	Template Upload Figma design	41
B.1	Sprint 1 Burndown Chart	42
B.2	Sprint 2 Burndown Chart	43
B.3	Sprint 3 Burndown Chart	43
B.4	Sprint 4 Burndown Chart	43
B.5	Sprint 5 Burndown Chart	44

Plagiarism Declaration

I affirm that the work I am submitting is entirely my own and has not been taken from any other source, except where properly cited. I understand that plagiarism, collusion, and copying are serious offences that can result in academic penalties. I have read and understood the assignment regulations and have properly cited all sources of information in my work. This assignment has not been previously submitted by me or anyone else for assessment on this or any other course.

List of Abbreviations

AWS	Amazon Web Services
EC2	Elastic Compute Cloud
ALB	Application Load Balancer
ASG	Auto Scaling Group
TG	Target Group
VPC	Virtual Private Cloud
SG	Security Group
S3	Simple Storage Service
VNC	Virtual Network Computing
IAM	Identity and Access Management
SSH	Secure Shell
HTTP	Hypertext Transfer Protocol
VM	Virtual Machine
IDE	Integrated Development Environment
API	Application Programming Interface
JWT	JSON Web Token
CDK	Cloud Development Kit
DNS	Domain Name System
PDF	Portable Document Format
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
AMI	Amazon Machine Image
BIOS	Basic Input/Output System
LXD	Linux Container Manager
SDK	Software Development Kit
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CI/CD	Continuous Integration and Continuous Delivery/Deployment
RBAC	Role Based Access Control

Acknowledgements

I would like to express my sincere gratitude to my project supervisor Jimmy McGibney for his guidance and support throughout the development of this project. His feedback and encouragement were invaluable throughout the process. I would also like to thank my friends and family for their continued support and encouragement throughout my final year.

1 Introduction & Background

1.1 Introduction

LearnSec is a cloud-based system for deploying virtual environments for digital forensic and cybersecurity classroom labs. This platform was developed to address the challenges that students face when setting up and accessing VMs. LearnSec leverages AWS cloud infrastructure to allow students to have on demand lab environments, pre-configured, that can be launched and connected to within the web browser. This eliminates the need for any installation of software or configuration of VM's.

This platform was designed for the purpose of using AWS Academy. The students can use their institution provided accounts, ensuring no unexpected costs and unnecessary resource usage. There are two types of roles for the platform, student and educator. The educator has access to all functionality, uploading and deleting lab instructions and launch templates. Students only have the ability to view lab instructions, create and launch instances from launch templates.

1.2 Background

A fundamental method of learning cybersecurity concepts for students education is through the use of VM's. This provides for example isolated environments to perform penetration testing, incident response and forensic analysis exercises. The traditional approach to setup these exercises was to install virtualisation software such as VirtualBox or VMware and setup the VMs through the software manually. This can lead to errors such as downloading the incorrect image files, misconfigured network settings, hardware limitations or allocating insufficient resources to the VM's. This creates a situation where students spends a large portion of time on setup and troubleshooting rather than completing the learning objectives.

There are other commercial cloud based solutions such as TryHackMe and Hack The Box which provides VM's in browser with lab instructions which mitigates the need for any manual configurations or installation. This is a clear indicator that there is a way to eliminate the possibility for human error through cloud hosting. These platforms are the inspiration to make an institution based version where educators have the ability to tailor content around a specific module. LearnSec was developed to bridge this gap.

LearnSec provides a platform with pre-configured lab environments available to students that can launch and connect to with a single click. This eliminates both human error and hardware limitations. Every student has access to identical virtual environments through their browser. This means all lab sessions can be accessed on demand regardless of their device or experience with VM's.

1.3 Motivation

The motivation for the development of LearnSec was by the deep interest in resolving the difficulties that students face when undergoing practical lab exercises using VM's. This was driven by personal experience with the limitations of traditional local virtual machine setups and the identification of a gap for an institution specific solution that gives educators full control over the content available to their students.

These factors combined creates a compelling motive to develop focusing on accessibility, consistency and a stress free experience for using VM's to learn cybersecurity concepts. The platform addresses both the technical and educational gap. Students were wasting time for their practical sessions because of setup and configuration issues and those with insufficient hardware were disadvantaged.

1.4 Problem Statement

Currently, there is no institution specific platform providing pre-configured VM's accessible through a browser to complete cybersecurity lab exercises. Instead, the traditional approach is still the standard accepted today which is leaving students spending excessive time troubleshooting and others at a disadvantage due to

hardware limitations. This creates an unbalanced learning experience. LearnSec aims to modernise the standard approach making a solution that provides VM's through a browser that are accessible, consistent and easy to use for all experience levels.

1.4.1 Industry Example

LearnSec was developed in the context of higher education however the platform has clear applications across a range of industry settings where practical hands-on training in cybersecurity and digital forensics is required.

Large organisations that are onboarding new security analysts require employees to develop skills with penetration testing tools and incident response procedures before they are given access to live production systems. LearnSec provides a browser based environment where users can practice these skills using pre-configured VM's without any risk to existing infrastructure.

Technology companies onboarding new software developers require an understanding of secure coding practices and common vulnerabilities before developers are given access to production code bases. LearnSec allows organisations to deploy pre-configured environments containing vulnerable applications for developers to identify and exploit. This helps to teach new hired about cybersecurity during the onboarding process.

Banks need to ensure their internal security teams are prepared to respond to threats such as ransomware attacks and data breaches. LearnSec enables organisations to deploy simulated environments that mirror their infrastructure. This is allowing security teams to practice detection and response procedures in a safe isolated environment without any impact on live systems.

Healthcare organisations are required to protect sensitive patient records and must ensure their IT security staff are able to handle and investigate data breaches. LearnSec provides a platform where staff can practice forensic investigation and incident response procedures in controlled environments before handling real incidents involving sensitive data.

1.5 Project Contributions & Impact

Throughout the development of LearnSec there was challenges in regards to finding documentation and examples of programmatically integrating Apache Guacamole, Python Flask and AWS services specifically EC2 instances. There was another issue regarding how AWS Academy learner lab sessions time out after four hours. This causes AWS services such as EC2 instances to stop after the four hour period.

To address this a AWS Free Tier account was used to host two S3 buckets one for launch templates one for lab documentation and AWS Cognito for user management. AWS Academy was used to host an EC2 instance containing the Flask web application and Apache Guacamole. This EC2 instance is kept running constantly through an ASG setting the minimum and maximum amount of instances to one. This means that at the end of the four hours and the learner lab times out the instance that contained the website is stopped but a new one is started to take its place. To ensure that this platform provides value to others the full source code is available on GitHub along with supporting markdown files detailing the application itself, the architecture, Apache Guacamole, the full deployment process and references.

2 Aims & Objectives

2.0.1 Aim

This project aims are to design and develop a website that allows educators and students to configure and deploy virtual environments through a browser using AWS services and Apache Guacamole, therefore eliminating the need for local installation.

2.0.2 Objectives

- Develop a web based application using Python Flask that allows students and educators to register, login, and access the platform through a browser.
- Allow educators to create and manage launch templates containing pre-configured VM specifications and assign them to specific class groups.
- Allow educators to upload and manage lab documentation organised by category.
- Allow users to launch VM's from available templates using their AWS Academy credentials.
- Integrate Apache Guacamole to provide users with browser based remote desktop access to their launched VM's with no additional software required on the user's device.
- Provide users with a dashboard displaying their running instances, recent lab activity and timer for expiration of AWS Academy credentials.
- Deploy the platform on AWS Academy infrastructure with an ALB, TG and ASG to ensure continuous availability.

3 Literature Review

Khan and Mohamed [2025] identifies and compares the effectiveness of cloud based virtual labs opposed to the traditional approach of using on premise environments for cybersecurity training. To test this specific exercises like penetration testing and vulnerability assessments were performed on both AWS EC2 and Virtual Box. The results showed that AWS EC2 outperforms Virtual Box in terms of resource efficiency and task completion speed. Khan and Mohamed [2025] highlights that using cloud infrastructure offers scalability and remote accessibility allowing for students to complete cybersecurity simulations without the limitations of local hardware. While using cloud infrastructure has proven to improve learning experience Khan and Mohamed [2025] makes note of certain hardware restrictions regarding hardware level network interactions and internet dependencies.

In conclusion, this research article clearly shows the benefits for the digital transformations of cybersecurity methods of practical education to better prepare students for evolving global threats. This results of this research article helped to shape the development of LearnSec, which leverages AWS EC2 eliminating hardware limitations to provide students with consistent and accessible lab environments. Khan and Mohamed [2025] requires students to create and configure their own EC2 instances manually LearnSec automates this process entirely. This allows for a seamless experience for students to launch and connect to pre-configured VMs through LearnSec's interface.

ben Othmane et al. [2013] investigates two universities implementation of hands-on lab environments into their cybersecurity curriculum moving away from theoretical knowledge and towards practical application. ben Othmane et al. [2013] demonstrates how students can safely practice cybersecurity attack and defence scenarios through using tools such as virtualisation platform, network simulators and penetration testing software. The main focus is the importance of isolating these labs from broader institutional networks to prevent the accidental spread of malware while allowing for testing of vulnerabilities. ben Othmane et al. [2013] focuses on providing a tool for which cybersecurity education allows for deeper understanding of fundamentals concepts and critical problem solving skills.

These results show the difficulties of managing locally hosted lab environments and directly supporting the motivation for LearnSec. By replacing the manual local setup with pre-configured VM templates that students can launch and connect to through a browser. This ensures availability and consistency for all students regardless of their hardware or technical background.

Franc et al. [2013] presents DF-LAB, a virtual lab environment designed to teach digital forensic concepts through collaborative distance learning. By adopting a project based learning methodology this bridges the gap between theoretical and practical application. This allows for students to carry out investigations in a simulated environment that reflect real world scenarios. Franc et al. [2013] puts an emphasis on DF-LAB and how it overcomes the traditional need for expensive physical equipment and specialised lab facilities while fostering student interaction and engagement. Franc et al. [2013] demonstrates how introducing VM into the curriculum helps to train future professionals in digital evidence acquisition.

This supports the educational approach taken in LearnSec, which aims to remove the need for physical lab infrastructure. DF-LAB requires significant infrastructure to deploy, whereas LearnSec builds on existing AWS Academy accounts already available to students using automated EC2 launch templates to provision pre-configured VMs on demand without any manual setup requirements.

Roslan [2026] talks about the development of a classroom cybersecurity lab platform specialised for education and designed to provide high performance hands-on training for students at Politeknik Mersing. A custom Moodle integration plugin was created and successfully moved the heavy computational work of running VMs from students devices to a centralised self-hosted private cloud. This removes hardware limitations

and financial barriers while also granting instructors control and realtime monitoring capabilities within a learning management system. Roslan [2026] also discussed the commerical platforms like TryHackMe and Hack The Box but this also limits educational controls for instructors and cloud platforms like AWS which can pose unpredictable costs and management overhead.

These findings relate to the motives for the development of LearnSec providing a seamless experience for launching pre-configured VMs. Unlike this particular classroom cybersecurity lab platform which focuses on using on-premise server infrastructure LearnSec uses AWS Academy accounts provided by the institution. This eliminates the need for additional hardware and still provides the same accessibility and consistent lab experience.

Justice and Vyas [2017] introduces RunLabs, a tool built to solve the recurring problem of creating virtualised cybersecurity lab environments that can be used within a single practical session. Manually creating these types of environments is time consuming RunLabs takes this process by allowing educators to define VMs in a JSON configuration file that can be deployed instantly. This allows for students access to their own isolated machines with VNC access. As part of Justice and Vyas [2017] future work it is recognised thar browser based access without any local installation would significantly improve the user experience.

This has been proven in LearnSec through the integration of Apache Guacamole. This provides a fully browser based remote desktop access to students EC2 instances without requiring any local installation. The JSON configuration file approach in RunLabs is adopted in the launch templates as they are saved in JSON format in a S3 bucket and used to create instances on demand through the platform.

Tóth and Erdődi [2023] conducted a comparative analysis of offensive security training platforms TryHackMe, Hack The Box, HackerOne, PicoCTF and PortSwigger Academy. Each platform was assessed on the following:

- Difficulty Levels.
- Ranking Systems.
- User Feedback.
- Extensibility.

The results of this analysis show the platforms provide engaging and accessible self-directed learning experiences but all have a significant limitations which are the inability for personalised learning pathways, adapting to users unique skill levels or institutional curriculum requirements. This was noted in future improvements as there is a gap for a platform which resolves around a structured academic delivery rather than individual skill building.

This directly supports the motivation for the development of LearnSec, to create a platform where educators have full control of the VMs templates and practical lab material with the ability to assign material to specific class groups. LearnSec allows for educators intervention whereas platforms like TryHackMe and Hack The Box restrict this type of functionality.

Hassan [2022] discusses how the implementation of secure and flexible virtual labs provided 250 undergraduates with hands-on cybersecurity training during the COVID-19 Pandemic. By combining Apache Guacamole, Docker and LXD containers Hassan [2022] described how a clientless system was created which allowed for access to Linux terminals through a web browser. This avoids hardware limitations Hassan [2022] described students with less than 8GB of RAM cannot run multiple VMs concurrently, virtualisation may be disabled in the BIOS and Apple M1 laptops are unsupported. As a result of the survey, Hassan [2022] shows that 88% of participants agreed that the solution was flexible and highly valued the ability to access an environment where they can practice their cybersecurity skills.

Hassan [2022] development and results show how it is important for LearnSec as it shows the real world impact of using Apache Guacamole as a browser based gateway for students to access lab environments. LearnSec provides students with remote desktop access to EC2 instances. Hassan [2022] solution requires on site infrastructure. LearnSec achieved the same browser based accessibility through AWS Academy and Apache Guacamole removing the need for any institutional hardware.

4 Architecture & Design

4.1 Architecture Model

The AWS architecture diagram in 4.1 shows how both AWS accounts are used for this platform.

The AWS free tier account is responsible for the persistent services. It creates and hosts two S3 buckets one for lab instructions, one for launch templates and AWS Cognito for user management. This is all done through CDK's .

The AWS Academy account is responsible for the hosting of all other compute infrastructure within a VPC containing a public subnet. There is an ALB is directing incoming traffic to two TG's. The TG WebFlaskLabASG-1 on port 8081 which is routing traffic to the Flask web application and WebGuacamoleTG-1 on port 8080 which is routing traffic to Apache Guacamole. Both the Flask web application and Apache Guacamole are running on an EC2 instance is managed by an ASG which is set instance amount as a minimum and maximum as one ensuring constant uptime.

The SG that is associated with the EC2 instance has allowed inbound traffic from the following ports:

- **Port 22** : SSH access to the EC2 instance for deployment and maintenance.
- **Port 80** : HTTP traffic allowing users to access the platform through a web browser.
- **Port 5901** : VNC connection to student EC2 instances.
- **Port 5902** : VNC connection to student EC2 instances.
- **Port 8080** : Apache Guacamole web interface.
- **Port 8081** : Flask web application.

The SG that is associated with the EC2 instance has allowed all outbound traffic.

When the user accesses the platform the requests goes to AWS Cognito verifying the user has an account. Once the user is confirmed they're routed through the ALB directing them to the choose role page to login to the Flask web application.

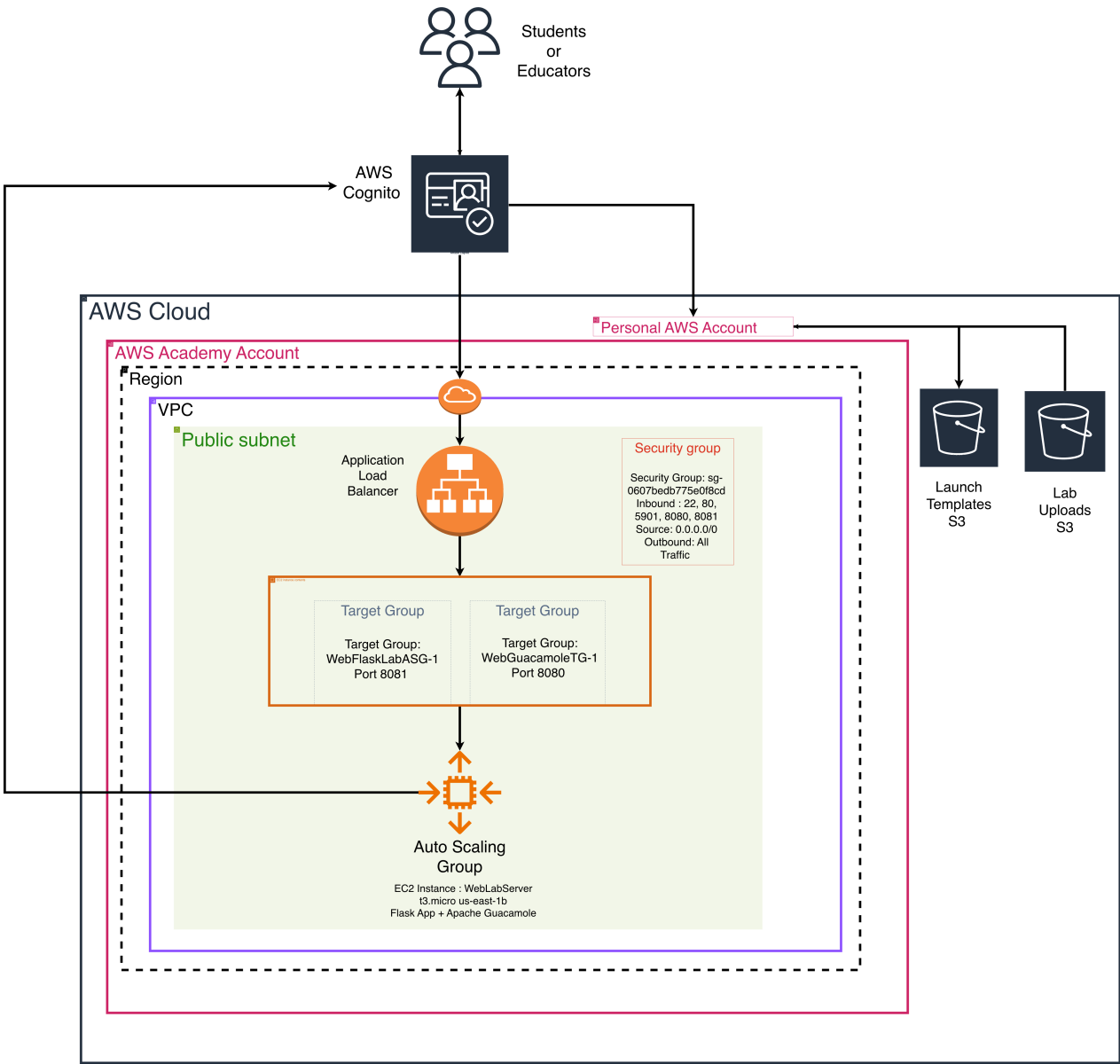


Figure 4.1: LearnSec Architecture Diagram

4.2 Design of Website Pages

This section outlines the design of each page within the LearnSec platform. For each page a description of its purpose and functionality is provided along with the key components and technologies used in its implementation. The pages covered in this section are Registration, Confirmation Code, Choosing Roles, Login, Account, Labs Documentation, Dashboard, Templates, and Launching Instances from Templates. The design of the website using Figma and the website pages itself are found in A.1

4.2.1 Registration and Confirmation

The registration page allows for new users, student or educator to create an account on LearnSec. The registration page requires the following information:

- **AWS Account ID** : collected to identify and associate the user with their AWS account within the platform.
- **Email Address** : used to automatically determine the user's role based on the email prefix. The user does not need to manually select their role during registration (Student is numerical & Educator is alphabetical). The email address is also used by AWS Cognito to send a confirmation code to verify the account. The only emails allowed to be used are restricted to institution email addresses.
- **Class Group** : collected to filter the content visible to the student, ensuring they only see content relevant to their module.
- **Password**: required to secure the user's account.
- **Confirm Password** : ensures the password was entered correctly before the account is created.

To determine the type of user which can be (Student or Educator) their email prefix is used to identify their role. If the email contains numerical values it is a student and if it is alphabetic it is an educator. The registration form validates all fields on submission and restricts registration to only institution email addresses (@setu.ie and @mail.wit.ie).

The confirmation page is shown once the registration form has been completed successfully. This shows the users email address and a blank text box where they enter a confirmation code sent to them via the email address they used to register. This is how AWS Cognito's confirms a user.

4.2.2 Components of Registration and Confirmation Page

Amazon Cognito

Amazon Cognito is a service in AWS used for identification of a user for mobile and web applications. It is composed of a user directory, an authentication server, and an authorisation service for OAuth 2.0 access tokens and AWS credentials. Amazon Web Services [2024d]

In this project Amazon Cognito authenticates and authorises users accessing the web application and creates a user pool which results in two user groups: Student and Educator.

User Pools

A user pool is a user directory provided by AWS Cognito that manages user registration, management and authentication, issuing a JSON Web Token to the application upon successful login which is then used to verify the user's identity throughout their session. Amazon Web Services [2024d]

The registration process involves the user receiving an email to the email address that was used in the registration form with a confirmation code. The user must enter in the confirmation code to change the status of their account from "unconfirmed" to "confirmed".

Within the application user pools are designed to store the users who have completed the registration process marking them as "confirmed" or "unconfirmed". The user will be assigned a user group via a Lambda function which looks at the custom:role attribute that was set during registration. If a user has started the registration process and not entered in the authentication code provided by AWS Cognito then the user will remain as "unconfirmed". This will result in the user not having access to the website and not being assigned to the appropriate user group until the user completes the registration process.

Amazon Web Services [2024f]

User Groups

User pools enables user groups to be created and managed. User groups are used to distinguish a collection of users to represent different types of users. This can then used to decide what permissions users in specific groups have. Amazon Web Services [2024e]

Two user groups are created in this application: Students and Educators. During registration if the prefix provided involves numbers only it is classified as a student and if it involves letters then it is an educator. A Lambda function then places the users into their appropriate groups based on their custom:role attribute that was set during registration. During the registration students and educators must choose a class group from the options which are DevOps, Cloud Computing and Network Security. This is then stored as a custom attribute within AWS Cognito allowing for more distinction between students and educators.

4.2.3 Choosing Roles

The choose role page is the first page the user is presented with when the user attempts to login. It shows two options for logging in student or educator. This is allowing the user to pick the correct login route. There is also a link to the registration page for users who have not made an account.

4.2.4 Login

The login page consists of two separate login routes one for students and one for educators. The login page requires the following information:

- **Email Address** : used to automatically determine the user's role based on the email prefix and identify they are registered. The user needs to manually select their role during login.
- **Password**: required to secure the user's account.
- **AWS Access Key ID** : a unique identifier for the AWS session
- **AWS Secret Access Key** : a secret key used alongside the Access Key ID to authenticate requests to AWS services
- **AWS Session Token** : a temporary token generated for the session that expires after four hours.

Since LearnSec interacts with AWS Academy to launch EC2 instances and describe running instances AWS Academy credentials must be provided.

4.2.5 Account

The account page is tailored to suit AWS Academy in regards to the ability to enter and re-enter temporary credentials into the system through three input fields:

- **AWS Access Key ID** : a unique identifier for the AWS session
- **AWS Secret Access Key** : a secret key used alongside the Access Key ID to authenticate requests to AWS services

- **AWS Session Token** : a temporary token generated for the session that expires after four hours

These credentials are attained from the learner lab console where the student starts the lab session. In the educator account page this is the same process credentials are directly stored in the Flask session without additional processing as they are temporary credentials.

Refreshing AWS Credentials

The credentials provided by the learner lab have a built-in expiration cycle of four hours. When the learner lab environment is either restarted or ended the credentials the students provided prior are now invalid. The student must re-enter the credentials again once the lab has been restarted with the newly generated values. To maintain secrecy there is a built in password type input for the secret access key but the session token and access key are visible. The account page has this functionality to re-enter the credentials which is essential for students to be able to continue using the website.

Amazon Web Services [2024a] Amazon Web Services [2024p] Pallets Projects [2024b]

4.2.6 Labs Documentation

The lab documentation page displays all the practical lab material available to users. The lab documents are categorised as follows:

- Networking.
- Databases.
- Security.

There is a drop down menu feature allowing for the selection of a topic and it displays all material associated with the topic. Each lab shows the document name, file size and a view button allowing for the user to open the PDF in the browser. Additionally, educators are able to delete each lab through a button function similar to the view button.

4.2.7 Components to Lab Documentation Page

S3 Storage Setup

The project uses Amazon S3 to store lab PDF files this provides reliability, scalability and storage allowing for multiple users to access files simultaneously. The S3 bucket was set up through a CDK which ensures consistent deployment and maintaining the same configuration whenever it is deployed.

The S3 bucket is configured for public read access which is necessary for this educational context. Since the students use AWS Academy accounts with temporary credentials it is vital that the S3 bucket is publicly available focusing on accessibility over strict access control.

Instead of using separate folders the application uses S3 object key prefixes to create clear structure. When a PDF is being uploaded the educator selects a category the file is then stored like for example **"Networking/lab1.pdf"** which is treated as if it's in a folder by the S3 Bucket.

The Flask application is able to query the specific category using the `list_objects_v2` operation with prefix filtering. The application can extract all the PDFs in any category without having to inspect the entirety of the S3 bucket.

Amazon Web Services [2024o] Amazon Web Services [2024k] Amazon Web Services [2024c] Amazon Web Services [2024m] Amazon Web Services [2024s] Amazon Web Services [2024l] Amazon Web Services [2024b]

File Upload Process

The file uploads process adopts a common workflow where the educator adds new lab materials. The user interface for uploading labs is only accessible to the educators. When an educator selects a PDF and a category for that file the upload process uses boto3's `upload_fileobj` method to push the file to the S3 bucket. The filename is sanitised before upload using the `secure_filename` function from Werkzeug to prevent any issues with user provided file names. This approach is memory efficient than loading the entire file into memory before uploading which is important when dealing with larger PDF files.

Amazon Web Services [2024v] Pallets Projects [2024c] Pallets Projects [2024a]

Viewing Labs

The lab viewing system works by displaying all labs material in their respective sections or using the drop down menu to query the S3 for the files in each category which results in them being presented in the filtered interface. The JS filtering lets users to effectively refine the content and navigate without additional server requests results in improved user experience.

When students want to view a lab the system creates a URL that provides secure temporary access to the PDF files. These URLs are configured with special parameters that make the PDFs open directly in the browser for easy reading rather than forcing a download. The URLs expire after one hour which allows for plenty of time to review the material while ensuring the links don't remain active indefinitely. This approach works well for the educational setting allowing students access to read the content while the temporary nature of the links maintains security without restrictions.

Amazon Web Services [2024n] Amazon Web Services [2024t] Mozilla Developer Network [2024]

Educator Management Features

Educators have the ability to delete the PDFs this process includes role verification and JS confirmation dialog to prevent accidental removal of files. This simplifies content management and consistent access control on user roles. Amazon Web Services [2024q]

4.2.8 Dashboard

The dashboard is the main page of the platform presented to the user upon successful login. It provides an overview of the user's current session and AWS resources through a series of cards.

The AWS Academy Session Timer card displays a live countdown of the remaining time in the current Academy session calculated from the login time stored in the Flask session. The timer changes colour based on the time remaining green when the session is active, yellow as a warning, and red when the session is close to expiring. It provides a direct link to the account page to refresh credentials when needed.

The Quick Stats card displays the user's role, class group, and the time their session started.

The Instance Overview card shows the total number of EC2 instances associated with the user's AWS credentials which broken down into running and stopped counts.

The My Instances card displays a table of all instances with their name, type, state, and launch time, with a Connect button available for any instance that is currently running.

The Recent Activity card displays the last five lab documents the user has viewed or launched, logged automatically each time a lab is accessed.

4.2.9 Launch Templates

The Templates page displays all VM launch templates available to the user. Templates are stored as JSON files in the S3 bucket and are filtered based on the user's class group meaning students only see templates assigned to their class or templates marked as available to all classes.

Each template entry displays the template name, description, and file size, with a Launch button that triggers the creation of an EC2 instance using the template's configuration. Educators are additionally presented with Edit and Delete buttons for each template allowing them to manage the available templates on the platform.

4.2.10 Components of Creating Launch Templates Page

Launch Template Infrastructure

The templates infrastructure creates an S3 bucket which is dedicated to storing launch templates this follows the same creation process as the labs systems S3 bucket but the main difference is storing launch templates JSON files. The bucket uses a policy of public read access to ensure availability for AWS Academy accounts ensuring students can access template configurations.

The public read policy allows any AWS user to access template files which is essential for the educational context where students use various AWS Academy accounts with temporary credentials. Amazon Web Services [2024o] Amazon Web Services [2024k] Amazon Web Services [2024c] Amazon Web Services [2024p] Amazon Web Services [2024l]

Template Creation and Organisation Process

Educators and students are able to create launch templates but are required to provide the following pieces of information:

- **Template Name** : required to identify the template on the platform and tag the EC2 instance when it is launched.
- **Share to Class Group** : required to restrict the template to a specific class group ensuring students only see templates relevant to their module.
- **AMI ID** : required to specify which pre-configured VM image to use when launching the instance.
- **Instance Type** : required to define the computing resources allocated to the VM.
- **User Data** : required to provide a script that runs automatically when the instance launches.
- **Description** : an optional field allowing additional context about the template to be provided.

When a launch template is created it is saved in JSON format. Information such as EC2 configuration details such as AMI ID and instance type are saved. The application also includes information such as the email address associated with the template. This means that whoever creates the template all the information for launching instances is recorded. Amazon Web Services [2024i] Python Software Foundation [2024b] Amazon Web Services [2024m] Amazon Web Services [2024g] Amazon Web Services [2024c]

4.2.11 Launching Instances from Templates

Template Viewing Interface

The template available page displays all available templates. All the essential information about a launch template is displayed including the template name and size along with the ability to launch an instance using that specific template. Educators get additional option to edit and delete a template with JavaScript confirmation dialog to prevent accidental removal of templates that other educators or students might be using.

The interface will show no templates available if the S3 bucket contains no launch templates.

Selecting & connecting to a Launch Template through an EC2 Instance

The launching of a template to create an EC2 is started directly from the templates page when a user clicks the Launch button on an available template. The Flask application retrieves the JSON file from the S3 templates bucket and extracts the AMI ID, instance type, and user data script.

Using the users' AWS Academy credentials stored in their session the application calls EC2 to create a new instance with the launch template. If a user data script is present in the template Boto3 automatically formats it to be base64 encoded before being passed to EC2 as required by AWS. The launch template name is tagged for easy identification and the user is immediately redirected to the instances page where the newly created instance appears with a pending status while AWS initialises it.

Once the instance changes from pending to a running state a Connect button becomes available which is allowing the user to proceed to connect to their virtual machine. Amazon Web Services [2024i] Amazon Web Services [2024o] Python Software Foundation [2024b] Pallets Projects [2024b] Amazon Web Services [2024u] Amazon Web Services [2024h] Amazon Web Services [2024g]

4.2.12 Components to allow for connection to instances

The instances page lists all EC2 instances in the student's AWS Academy account along with their name, instance type and current state. The connect button is only displayed for instances in a running state, ensuring students cannot attempt to connect to instances that are still initialising or have been stopped. When a student clicks the connect button the application handles the entire connection process in the background it gets the instance's public IP address and creating a VNC connection in Guacamole. This then redirects the student to their VM through the Guacamole client in the browser. Amazon Web Services [2024h] Pallets Projects [2024b] Amazon Web Services [2024u] Amazon Web Services [2024r] Amazon Web Services [2024j]

Apache Guacamole

Apache Guacamole connects the student's browser and their running EC2 VM's. When the connect route is triggered the Flask application gets the instance's public IP address from EC2 and authenticates through the Guacamole API running internally on the same server. It removes any existing duplicate connection before registering a new VNC connection pointing to the instance on port 5901. The connection identifier returned by Guacamole is base64 encoded in the format required by the Guacamole client URL and the student is redirected directly to their session. This bypassing the Guacamole login screen entirely. The virtual machine desktop is then rendered as an HTML5 canvas in the browser with all keyboard and mouse input passed back through the Guacamole daemon to the VNC server running on the student's instance providing a fully interactive lab environment accessible entirely through the browser.

Apache Software Foundation [2024] ridvanaltun [2024a] ridvanaltun [2024b] atomlab [2024] Python Software Foundation [2024c] Python Software Foundation [2024a] Amazon Web Services [2024r] Amazon Web Services [2024j]

4.3 User Stories

- As a student, I want to register for an account using a university email so that I'm automatically put in the correct user group.
- As a student, I want to enter my AWS Academy temporary credentials so that I can access and launch VMs for my lab sessions.
- As a student, I want to browse and view available lab documents by category so that I can look at relevant instructions for each module.
- As a student, I want to launch a VM from an available template so that I can complete practical lab material without manual VM setup.
- As a student, I want to access a launched VM through the browser using Apache Guacamole so that I don't have to download additional software.
- As an educator, I want to upload lab instruction PDFs and assign them to a category so that students can easily find and access the correct materials.

- As an educator, I want to delete outdated lab documents so that students are not accessing incorrect or irrelevant materials.
- As an educator, I want to create launch templates with specific AMI IDs and instance types so that students can launch pre-configured VMs suited to the lab.

4.4 Risks & Mitigations

A number of risks were identified during the planning and development of LearnSec.

A significant risk that was uncovered in the development of LearnSec which was the limited documentation on Apache Guacamole and integrating it with the Python Flask application and EC2 instances. This raised concerns about automating the browser based connection and it could potentially not be feasible within the timeframe provided. To mitigate this risk Apache Guacamole was researched extensively before implementation began. If the integration of Apache Guacamole into a Flask application was not possible the students would be directed to the Apache Guacamole login interface or the previous prototype solution developed in Semester one would be used instead.

The prototype from semester one implemented port forwarding through the Flask web application instance to tunnel VNC connections from students directly to their VMs running on private subnets. This approach demonstrated how TightVNC was installed on the EC2 instances and VNC Viewer was used on the host machine to connect through a forwarded local port.

AWS Academy has unique features that were identified as risks. Firstly, AWS Academy Learner Lab sessions timeout after four hours. This causes services like EC2 instances to stop immediately. This means that there is a risk to the availability of the platform as it is hosted on an EC2 instance. This risk has been mitigated by creating and configuring an ASG with a minimum and maximum of one instance ensuring that the platform is constantly available. When one instance is stopped another is created to replace it.

The dual AWS account architecture introduced a risk that cross account access between the AWS Free Tier account hosting S3 buckets, AWS Cognito and the AWS Academy account hosting the EC2 instance would not function correctly due to permission restrictions. This was mitigated through careful CDK configuration and bucket policy management to ensure the AWS Academy account could access the S3 buckets containing lab material and launch templates.

The risk of not completing all planned features within the project timeline was mitigated through following Agile methodology and sprint based development ensuring the most critical features were prioritised and delivered first.

5 Implementation Methodology

5.1 Methodology

This project followed an iterative development approach where the system was developed in incremental stages over multiple sprints. Each iteration focused on implementing specific features, testing functionality and refining the system based on observed issues and requirements.

5.1.1 Agile & Scrum

Agile methodology is a type of approach where work is divided into phases and completed continuously. This emphasis continuous delivery and improvement.

Atlassian [2025a]

An Agile approach was taken to allow for continuous development.

Scrum is an Agile framework used to help teams structure and manage their work effectively. It is based on values, principles and practices that promote learning through experience. This encourages self organisation and continuous improvement through reflection on successes and failures.

Atlassian [2025b]

Elements of Scrum were used to organise development into manageable cycles allowing features to be developed, tested, and refined in stages.

Rather than strictly following all formal Scrum practices selected principles such as iterative development, incremental delivery, and continuous feedback were applied to suit the scope of the project.

5.1.2 Scrum Artifacts

Scrum artifacts provide insight into the performance of a sprint and are essential to teams as they support the characteristics of Scrum which are transparency, inspection and adaptation.

PM [2024]

Trello was used as the primary tool for the management of Scrum artifacts. A task backlog was maintained as a Trello board containing all planned features, improvements and issues organised into lists representing their current status. Tasks were ticked off as development progressed which provides a clear visual overview of outstanding and completed work at any point during the project.

5.1.3 Sprints, Sprint Review & Backlog Refinement

In project management a sprint is a short period during which a team focuses on completing a specific set of tasks and objectives.

KnowledgeHut [2024]

The development of the platform was spread across five two week sprints. Each sprint focused on a defined set of features which were implemented, tested, and reviewed.

At the end of each sprint a review process was carried out to assess the functionality implemented and identify any issues. This helped determine whether the sprint objectives had been achieved and informed planning for the next iteration.

Throughout the project, tasks were updated and given priority based on progress and requirements. This meant that larger tasks were broken down into smaller components to ensure they would be completed within a sprint timeframe.

5.1.4 Burndown Charts

A burndown chart is used in Agile project management to track a team's progress through a sprint. It shows the total effort planned against the amount of work remaining for each sprint. This provides clarity on whether the task is progressing at the expected rate throughout the sprint.

ProjectManager [2024]

Burndown charts were used to represent progress within each sprint by illustrating the reduction of remaining tasks over time. These charts provide a visual indication of development progress and help highlight whether work was completed within the expected timeframe.

These charts provide a visual indication of development progress and help highlight whether work was completed within the expected timeframe. Story points were assigned to each task based on estimated complexity and effort these tasks were all planned out in using Trello. At the end of each sprint the remaining and completed story points were used to generate burndown charts using the Teamcamp Burndown Chart Generator. Teamcamp [2024]

The burndown charts for each sprint can be found in Figures B.1 to B.5

5.2 Sprint 1

- Set up the new project repository, initial folder structure and clone last semesters repository as a starting point.
- Removed the previous stack and restarted the application environment.
- Rebuilt the base website and project configuration.
- Created Figma designs to plan the structure and layout of the website pages.
- Began development of core authentication features including login, registration, and account pages.
- Implemented AWS Cognito for user management which including student and educator user groups.

5.3 Sprint 2

- Continued development of individual website pages.
- Implemented lab upload and lab viewing functionality using Amazon S3.
- Created S3 buckets for storing lab materials and structured them using categories.
- Began development of launch templates functionality for provisioning instances.
- Developed the initial structure for both labs and templates features within the application.

5.4 Sprint 3

- Transitioned from AMI sharing to launch templates due to AWS Free Tier and AWS Academy restrictions.
- Developed and refined the launch templates page and sharing functionality.
- Improved existing pages which including adding class group selection to the registration page.
- Improved both the user interface and backend logic.
- Progressed report writing alongside development to document completed work.

5.5 Sprint 4

- Developed instance management website page and functionality.
- Enabled launching of instances from shared launch templates.
- Implemented dashboard functionality including a learner lab session timer.

- Researched Apache Guacamole and its integration requirements.
- Integrated Apache Guacamole into the system.
- Added a connect feature allowing users to access instances through the browser via Guacamole.

5.6 Sprint 5

- Migrated the project from AWS Free Tier to AWS Academy.
- Updated the system to support AWS Academy credentials and environment constraints.
- Continued testing and resolving compatibility issues during the migration process.

5.7 Version Control

Version control is the practice of tracking and managing changes to software code throughout the development process. It is supported by version control systems which are tools that help teams manage and record modifications to their codebase over time ensuring a full history of changes is maintained.

Atlassian [2024a]

Version control was implemented to manage changes to the project codebase throughout development and also the development of this report. This was done through the use of GitHub and Overleaf's version control mechanism. This allowed for tracking modifications, maintenance of different versions of the system through branches and supporting iterative work practices.

5.8 CI/CD

Continuous Integration involves regularly committing and merging code changes into a repository. Continuous Delivery involves ensuring that any changes to code are tested and ready for deployment at any point during development. Together CI/CD aims to streamline the development process by maintaining a consistent and reliable code base throughout the project.

Hat [2024]

Continuous Integration practices were followed by regularly committing and pushing code updates to a GitHub repository. After each change the system was deployed and tested to ensure functionality was maintained.

Instead of an automated pipeline for continuous deployment this was done manually. When an update was made it was deployed and tested manually. This allowed for greater control over changes and making it easier to identify any issues before continuing development.

6 Tool & Technologies

6.1 AWS

AWS offers a wide range of on demand internet based services, such as databases, networking, storage, and processing power. AWS provided the persistent backend services used by the platform in this project.

Amazon S3 was used to store lab instruction PDFs and launch template configuration files which was organised across two dedicated buckets one for lab documents and one for templates. The lab documents S3 bucket can only be accessed by the educator but both student and educator can access the launch templates S3 buckets for both purposes of uploading and launching.

Amazon Cognito was used to manage user authentication and maintaining two user groups representing students and educators. The Flask application stores the user's role in the session and uses it to enforce RBAC for requests on the platform.

Services [2024b]

6.2 AWS Academy

AWS Academy offers educational institutions and students access to AWS cloud curriculum and practical lab environments.

AWS Academy Learner Lab was used to host the infrastructure for this project. The learner lab environment sessions lasts four hours once they are started. EC2 instances are stopped automatically once the four hours have passed or when the learner lab is terminated and they can be restarted when the learner lab is started. Services that are required consistently such as S3 buckets and AWS Cognito were hosted on a AWS Free Tier account. This due to the nature of the learner labs and so these resources are available whether the learner lab is active or not.

This behaviour directly influenced the architecture of the platform. To ensure the website was available consistently, an ASG was created with a minimum and maximum of one instance running. AWS Cognito and S3 buckets with launch template configurations and lab material were hosted on a free tier account to ensure they are available at all times and unaffected by the four hour session expiry.

Services [2024a]

6.3 Apache Guacamole

Apache Guacamole is an open source remote desktop gateway that supports standard protocols like VNC, RDP, and SSH. There is no need to install any additional client software as it can be accessed solely through a web browser. In this project Guacamole was integrated with the Flask application to enable students to connect to and interact with their running EC2 instances directly through the platform fulfilling the core requirement of providing a fully browser-based lab environment.

Foundation [2024a]

6.4 Overleaf

Overleaf is an online LaTeX editor and is the tool used to create this report. Overleaf allows the creation of professionally formatted documents using plain text instead of formatted text as typically done with other document writing tools like Microsoft Word.

6.5 Git and GitHub

Two essential tools for putting in place a practical version control system are Git and GitHub. GitHub is a cloud-based platform that offers repositories for tracking code base changes and Git is a command-line tool for committing to repositories through the terminal. The source code for this project was managed using Git

and GitHub which allowed work to be done on several features at once and allowed changes to be tracked and undone if problems occurred.

GitHub [2024]

6.6 PyCharm

Pycharm was used as the IDE for this project as it is designed for python development. It provides support for the Flask framework including Flask routes, Jinja2 templates and WTForms for the frontend and Boto3 to communicate to AWS for the backend operations.

Version control was managed throughout the project using Git and GitHub, both of which are natively supported within PyCharm. PyCharm's context-aware AI Assistant was used throughout the project to support productivity during development. This was especially beneficial when working with unfamiliar AWS SDK methods and Apache Guacamole configuration where the AI Assistant provided relevant suggestions that reduced the time spent consulting external documentation and allowed greater focus to be placed on the core functionality of the platform.

JetBrains [2024]

6.7 Trello

Trello is project management tool developed by Atlassian that allows users to organise tasks and workflows using a visual boards on a website. Trello was selected as the project management tool for this project. This tool is familiar to me since I've been using it for other modules. It provides a nice user interface representing tasks as cards and allowing for tasks to be ticked off as they are completed. It allowed me to set fixed deadlines to make sure work was completed throughout the development of LearnSec.

Atlassian [2024b]

7 Programming Languages & Frameworks

7.1 Python

Python is a widely used programming language known for its clean and readable syntax. It supports multiple programming models including object-oriented and procedural programming making it a flexible choice for a wide range of applications from web development to data science. Python's extensive standard library and broad collection of third party packages make it particularly well suited for rapid application development.

Foundation [2024b]

Python was the chosen backend language for LearnSec due to its flexibility and extensive support of AWS services through the use of Boto3 library and to create the web application through Flask web framework. All server side logic was written in Python and manages the following:

- Handling user registration and authentication.
- Uploading and retrieval of lab documents.
- Uploading and retrieval of launch templates.
- Launching of EC2 instances using AWS Academy credentials.
- Automated connection process through the Apache Guacamole.

7.2 Flask

Flask is a minimal Python web framework designed to make web application development straightforward and accessible. It provides a simple foundation for building web applications without enforcing any particular structure or requiring specific third party dependencies. Flask has grown to become one of the most popular Python frameworks due to its simplicity, flexibility and the large number of community maintained extensions that allow developers to add functionality as their application requirements grow.

Projects [2024a]

Flask was used as the web framework for LearnSec and manages the following pages:

- Registration.
- Login.
- Labs documentation.
- Launch templates.
- Dashboard.
- Instance management.

These were implemented as Flask routes.

Flask sessions were used to store temporary AWS Academy credentials and user information throughout each session. This ensures that the AWS Academy credentials are available across all routes without requiring the user to re-enter the credentials on each page they visit. RBAC was implemented by storing the user's role in the Flask session upon successful login and each route checking the session to determine whether the user was a student or educator and restricting access to pages accordingly. This ensured that educator only functionality such as uploading lab documents and managing launch templates was not accessible to students.

7.3 Boto3

Boto3 is the official AWS SDK for Python allowing developers to interact with AWS services directly from Python applications. It provides the ability to interact AWS resources including S3, EC2 and AWS Cognito. Boto3 handles the AWS API interactions including credential management, request signing and error handling. This allowed developers to focus on application logic rather than low level API details.

Services [2024c]

Boto3 was used throughout the LearnSec backend to interact with the following AWS services:

- Amazon S3 for uploading, retrieving and deleting lab documents and launch templates.
- Amazon EC2 for launching instances from template configurations describing instance state and retrieving public IP addresses for Guacamole connections.
- Amazon Cognito for the management of user authentication and retrieving user group information.

7.4 Jinja2

Jinja2 is a templating engine for Python that allows developers to build dynamic website pages by combining HTML with Python syntax. Templates can contain variables, loops and conditional statements that are evaluated and replaced with real data when the page is rendered. This allows the same template to display different content depending on the data passed to it.

Projects [2024b]

Jinja2 served as the templating engine for all HTML pages in LearnSec. It was used to dynamically populate pages with data retrieved from AWS services such as displaying available lab documents, launch templates and running instances. RBAC was implemented using Jinja2 logic ensuring that educator specific features such as file upload and delete buttons were only visible to users with the educator role.

7.5 HTML

HTML is the foundational language of the web. It is used to define the structure and meaning of web content forming the backbone of every web page. HTML works alongside CSS for visual presentation and JS for interactive behaviour to produce the complete web experience seen by users.

Network [2024b]

Throughout LearnSec, HTML was used to define the structure of every page presented to the user. Every page was built using HTML templates, where Flask and Jinja2 would show content depending on the user's role and session. HTML tables, forms, buttons and drop down menus were used across the pages to allow users to interact with the system.

7.6 CSS

CSS is used to style HTML pages, showing how they look when displayed in a web browser. This allows developers to control the appearance of web pages including text colour and size, page layout and visual effects. CSS applies visual styling to HTML elements through a set of defined rules and keeping the appearance of a page separate from its underlying structure.

Network [2024a]

In LearnSec CSS was used to style all HTML pages across the platform.

7.7 JavaScript

JS is a lightweight programming language that runs in the web browser and enables dynamic functionality to website pages. It works alongside HTML and CSS as the third layer of web development, adding the ability to update content in real time, respond to user events and create interactive experiences. Unlike server side languages which run on a server and JS runs directly on the user's device as client side code.

Network [2024c]

JS was used in two areas of LearnSec. In the labs document page JS handled the category drop down filtering. This allows for the display content to update instantly when a user selected a different category without needing to reload the page. On the dashboard, JS was used to implement the AWS Academy session countdown timer, updating the remaining time every second and changing colour from green to yellow to red as the session approached expiry.

8 System Testing & Evaluation

8.1 Testing Strategy

Software testing is an important component of development of a software application in order to meet the requirements for the functionality of the application. There are multiple approaches to test strategies for different types of software applications. For LearnSec three test strategies have been identified and considered which are static testing, structural testing and behavioural testing. BrowserStack [2024]

Static testing is analysing and evaluating the code before running the system. This involves reviewing code before running it to identify issues early on in the development process. This was done through assessing the code for each feature after it was written before committing the code to the GitHub repository. BrowserStack [2024]

Structural testing which is a technique of unit testing is the process of testing a feature after writing the code to identify errors. This is known as white-box testing which means that the features will be tested with the knowledge of the devices and systems that it is functioning on. In this instance each feature once it was developed and went through static testing it was important to look at how the features interact with AWS services through the application. BrowserStack [2024]

Behavioural testing focuses on workflow and how the system acts rather than the code. It focuses on how the feature performs and how the users experience will be effected. This can be referred to as black box testing which is concerned with the users perspective and not so how the features communicate with AWS services. It covers both user types which are student and educator and focuses on how for example when a launch template is created is it shown in the list of templates and can an EC2 instance be created from that template with no complications throughout. This helped to create the user stories discussed. BrowserStack [2024]

8.2 Testing Approach

The main testing approach adopted was the manual functionality testing which falls under the behavioural approach. As defined before this approach involves for testing each feature of the system from the both types of users perspective to verify that the feature is functioning as intended. This was the preferred approach over automated testing as it allowed me to see if there are any issues or errors for the users that will not allow for the application to work effectively. BrowserStack [2024]

8.3 Future Testing

As LearnSec is a student project rather than a commercial product, some testing strategies were not implemented but would be considered for future development. These include:

- **Unit Testing:** automated testing of individual code components in isolation using a framework such as pytest for Python to verify each function behaves correctly.
- **Performance Testing:** testing the platform under load to verify it can handle multiple students launching instances and connecting through Guacamole simultaneously.
- **Security Testing:** testing for common vulnerabilities such as SQL injection and cross site scripting to ensure the platform is secure.
- **Automated Regression Testing:** implementing automated tests to verify that new changes do not break existing functionality that was previously working.

8.4 Evaluation

From completing the behavioural testing LearnSec meets all the objectives that were set out at the start of this project. Students can register, login, browse lab documentation, launch EC2 instances from pre-configured templates and connect to their VM's entirely through the browser without any local software installation required. Educators have full control over the lab materials and virtual machine templates available to their students, with the ability to assign content to specific class groups. The dual AWS account architecture successfully overcame the limitations of AWS Academy by ensuring persistent availability of user data and platform infrastructure across sessions. Additional testing has been identified for the future of the LearnSec platform.

9 Future Product Developments

LearnSec successfully delivers a browser based virtual lab platform for cybersecurity and digital forensics education. However there are several features that could be developed further to enhance the platform's functionality and accessibility.

Moodle Plugin

A Moodle integration plugin similar to the one described by Roslan [2026] would allow students to launch EC2 instances directly from within their university learning management system without having to visit LearnSec separately. This would further reduce friction for students by keeping all course material and lab access in one familiar environment.

Multi-Cloud Support

LearnSec currently relies entirely on AWS Academy for compute infrastructure. Future improvements to the platform would be to support alternative cloud providers such as Microsoft Azure or Google Cloud. This would make LearnSec accessible to institutions that do not have access to AWS Academy and significantly broadening the range of institutions that could adopt the platform.

Automated AWS Academy Credential Refresh

Currently students must manually copy and paste three AWS Academy credentials every four hours when their session expires. Rather than the students having to check the timer on the dashboard to see when their AWS Academy credentials are expired, the platform could detect this automatically and prompt them to update their details when needed.

Notification System

A notification system that alerts students when new lab documents or launch templates have been added by an educator would ensure students are always aware of new material available to them without having to manually check the platform.

VM Snapshot Feature

A snapshot feature would allow students to save the current state of their VM and resume from that point in future sessions, rather than starting over each time. This would be particularly useful for longer lab exercises that cannot be completed within a single AWS Academy session.

Adaptive Lab Recommendations

Implementing a recommendation system based on student activity history would allow LearnSec to suggest relevant lab materials and templates to students based on what they have previously accessed. This would address the gap identified by Tóth and Erdődi [2023] regarding the absence of personalised learning pathways on existing cybersecurity platforms.

GitHub Repository Integration

Integrating GitHub into the launch template creation process would allow educators to specify a repository to be automatically cloned into a student's EC2 instance when it launches. This would be implemented through the user data script field that already exists in LearnSec's launch templates, requiring no significant architectural changes.

User Activity Logging

Implementing persistent user activity logging would allow educators to track which students have accessed which labs and templates, how long their instances were running and how frequently they are engaging with the platform. This would give educators greater visibility over student engagement and help identify students who may need additional support.

10 Conclusion

The development of LearnSec emerged from a motivation to create a solution for the way practical lab sessions were being delivered and the impact that had on students. The motivation, planning and development of LearnSec has been one of the most technically challenging and rewarding experiences of my time at South East Technological University. Throughout my degree, I have been exposed to a wide range of programming languages, tools, technologies and methodologies, but LearnSec has given me the opportunity to apply and enhance those skills in a manner that no previous modules have allowed me to before. Having the responsibility to plan and develop every component of the platform, from the AWS architecture to the user interface, posed both a challenge but gave me a greater understanding and appreciation to be involved in completing a project like LearnSec.

The one most challenging part was the integration of Apache Guacamole with the Flask application and AWS EC2 instances and turned out to be the most rewarding part of the development process of the project. This resulted in a workflow where when the connect route was triggered, the Flask application gets the instance's public IP address from EC2 and authenticates through the Guacamole API which runs internally on the same server. From here it registers a VNC connection to the instance, base64 encodes the connection to match the format expected by the Guacamole client URL, and redirects the student directly to their VM desktop in the browser.

The original approach of AMI sharing was replaced with launch templates stored in an S3 bucket in JSON format. This became necessary due to permission restrictions between the AWS Free Tier and AWS Academy accounts, and ultimately provided more flexibility by allowing user data scripts to be defined within each template. At the time it felt like a setback, but the resulting approach is more flexible and better suited to the educational context.

Another development change was made when moving to AWS Academy Learner Lab sessions instead of using AWS Free Tier accounts. AWS Academy Learner Lab sessions time out after four hours and stop services such as EC2 instances, this included the instance that was hosting the platform. The solution to this was to create an ASG which was to manage the platform and Apache Guacamole. This had a minimum and maximum instance amount of one ensuring that once the session had timed out and the instance supporting the platform and Apache Guacamole was stopped, a new instance was created automatically to ensure constant uptime.

Overall, LearnSec objectives have been completed. A web-based application was developed using Python and Flask to allow students and educators to register, login, and access the platform through a browser. Educators had the ability to create and manage launch templates with pre-configured VM's and can assign them to specific class groups the same is applied for lab material. Students can launch EC2 instances from the launch templates made available to the students through their AWS Academy account credentials. Apache Guacamole was implemented to provide browser based remote desktop access, requiring no additional software to be installed. A dashboard is available to display the users' running instances and recent lab activity along with other information such as AWS Academy credentials timer and account information. The platform remains accessible consistently through the deployment of an ALB, TG and ASG. The full source code is available on GitHub with supporting documentation, and the platform provides a solid foundation that could be extended and adopted by other institutions in the future.

Bibliography

- Amazon Web Services. Aws academy learner labs, 2024a. URL <https://docs.aws.amazon.com/academy/latest/userguide/learner-lab.html>. Accessed: 2026.
- Amazon Web Services. Aws cdk iam policystatement, 2024b. URL https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_iam/PolicyStatement.html. Accessed: 2026.
- Amazon Web Services. Aws cdk s3 bucket, 2024c. URL https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_s3/Bucket.html. Accessed: 2026.
- Amazon Web Services. What is amazon cognito?, 2024d. URL <https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>. Accessed: 2026.
- Amazon Web Services. User groups in amazon cognito, 2024e. URL <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-user-groups.html>. Accessed: 2026.
- Amazon Web Services. Amazon cognito user pools, 2024f. URL <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html>. Accessed: 2026.
- Amazon Web Services. Amazon machine images, 2024g. URL <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html>. Accessed: 2026.
- Amazon Web Services. Ec2 instance lifecycle, 2024h. URL <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-lifecycle.html>. Accessed: 2026.
- Amazon Web Services. Launch templates for amazon ec2, 2024i. URL <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-launch-templates.html>. Accessed: 2026.
- Amazon Web Services. Amazon ec2 instance ip addressing, 2024j. URL <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-instance-addressing.html>. Accessed: 2026.
- Amazon Web Services. Bucket policies for amazon s3, 2024k. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-policies.html>. Accessed: 2026.
- Amazon Web Services. Cross-account s3 access, 2024l. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-walkthroughs-managing-access-example2.html>. Accessed: 2026.
- Amazon Web Services. Using prefixes in amazon s3, 2024m. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-prefixes.html>. Accessed: 2026.
- Amazon Web Services. Using presigned urls in amazon s3, 2024n. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html>. Accessed: 2026.
- Amazon Web Services. What is amazon s3?, 2024o. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>. Accessed: 2026.
- Amazon Web Services. Temporary security credentials, 2024p. URL https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html. Accessed: 2026.
- Amazon Web Services. boto3 s3 delete_object, 2024q. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/delete_object.html. Accessed: 2026.

- Amazon Web Services. boto3 ec2 describe_instances, 2024r. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/client/describe_instances.html. Accessed: 2026.
- Amazon Web Services. boto3 s3 list_objects_v2, 2024s. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/list_objects_v2.html. Accessed: 2026.
- Amazon Web Services. boto3 s3 generate_presigned_url, 2024t. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/generate_presigned_url.html. Accessed: 2026.
- Amazon Web Services. boto3 ec2 run_instances, 2024u. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2/client/run_instances.html. Accessed: 2026.
- Amazon Web Services. boto3 s3 upload_fileobj, 2024v. URL https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/upload_fileobj.html. Accessed: 2026.
- Apache Software Foundation. Configuring guacamole for vnc, 2024. URL <https://guacamole.apache.org/doc/gug/configuring-guacamole.html#vnc>. Accessed: 2026.
- Atlassian. What is version control, 2024a. URL <https://www.atlassian.com/git/tutorials/what-is-version-control>. Accessed: 2026.
- Atlassian. Trello, 2024b. URL <https://trello.com/>. Accessed: 2026.
- Atlassian. Agile project management. <https://www.atlassian.com/agile>, 2025a. Accessed: 2026-04-05.
- Atlassian. Scrum - what is scrum and how to get started. <https://www.atlassian.com/agile/scrum>, 2025b. Accessed: 2026-04-05.
- atomlab. Guacamole rest api examples, 2024. URL <https://gist.github.com/atomlab/376901845c3d474d1e60e6b7a3affaae>. Accessed: 2026.
- Lotfi ben Othmane, Vijay Bhuse, and Leszek T. Lilien. Incorporating lab experience into computer security courses. In *2013 World Congress on Computer and Information Technology*. IEEE, 2013. URL <https://ieeexplore.ieee.org/abstract/document/6618731>.
- BrowserStack. Software testing strategies and approaches, 2024. URL <https://www.browserstack.com/guide/software-testing-strategies-and-approaches>. Accessed: 2026.
- Apache Software Foundation. Apache guacamole, 2024a. URL <https://guacamole.apache.org/>. Accessed: 2026.
- Python Software Foundation. What is python? executive summary, 2024b. URL <https://www.python.org/doc/essays/blurb/>. Accessed: 2026.
- Igor Franc, Zona Kostic, Aleksandar Jevremovic, and Ranko Popovic. Df-lab: Digital forensic virtual laboratory for collaborative distance learning. *Metalurgia International*, 18, 2013. URL <https://www.proquest.com/openview/21c44600581013391c98eff7a558ffff/1>.

- GitHub. About github and git, 2024. URL <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>. Accessed: 2026.
- Ismail Hassan. Leveraging apache guacamole, linux lxd and docker containers to deliver a secure online lab for a large cybersecurity course. In *2022 IEEE Frontiers in Education Conference (FIE)*. IEEE, 2022. URL <https://ieeexplore.ieee.org/abstract/document/9962510>.
- Red Hat. What is ci/cd?, 2024. URL <https://www.redhat.com/en/topics/devops/what-is-ci-cd>. Accessed: 2026.
- JetBrains. Pycharm, 2024. URL <https://www.jetbrains.com/pycharm/>. Accessed: 2026.
- Connie Justice and Rushabh Vyas. Cybersecurity education: Runlabs rapidly create virtualized labs based on a simple configuration file. In *2017 ASEE Annual Conference and Exposition*. ASEE, 2017. URL <https://peer.asee.org/cybersecurity-education-runlabs-rapidly-create-virtualized-labs-based-on-a-simple-configuration-file>.
- Adil Khan and Azza Mohamed. Optimizing cybersecurity education: A comparative study of on-premises and cloud-based lab environments using aws ec2. *Computers*, 14(8):297, 2025. URL <https://www.mdpi.com/2073-431X/14/8/297>.
- KnowledgeHut. Project management sprints, 2024. URL <https://www.knowledgehut.com/blog/project-management/project-management-sprints>. Accessed: 2026.
- Mozilla Developer Network. Content-disposition http header, 2024. URL <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Disposition>. Accessed: 2026.
- Mozilla Developer Network. What is css?, 2024a. URL https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/What_is_CSS. Accessed: 2026.
- Mozilla Developer Network. Html: Hypertext markup language, 2024b. URL <https://developer.mozilla.org/en-US/docs/Web/HTML>. Accessed: 2026.
- Mozilla Developer Network. What is javascript?, 2024c. URL https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/What_is_JavaScript. Accessed: 2026.
- Pallets Projects. Flask file uploads, 2024a. URL <https://flask.palletsprojects.com/en/2.3.x/patterns/fileuploads/>. Accessed: 2026.
- Pallets Projects. Flask sessions, 2024b. URL <https://flask.palletsprojects.com/en/2.3.x/quickstart#sessions>. Accessed: 2026.
- Pallets Projects. Werkzeug secure_filename, 2024c. URL https://werkzeug.palletsprojects.com/en/2.3.x/utils/#werkzeug.utils.secure_filename. Accessed: 2026.
- Nifty PM. Scrum artifacts, 2024. URL <https://niftypm.com/blog/scrum-artifacts/>. Accessed: 2026.
- ProjectManager. Burndown chart: What is it?, 2024. URL <https://www.projectmanager.com/blog/burndown-chart-what-is-it>. Accessed: 2026.
- Pallets Projects. Flask, 2024a. URL <https://pypi.org/project/Flask/>. Accessed: 2026.
- Pallets Projects. Jinja, 2024b. URL <https://jinja.palletsprojects.com/en/stable/>. Accessed: 2026.

- Python Software Foundation. base64 - base16, base32, base64 data encodings, 2024a. URL <https://docs.python.org/3/library/base64.html>. Accessed: 2026.
- Python Software Foundation. json - json encoder and decoder, 2024b. URL <https://docs.python.org/3/library/json.html>. Accessed: 2026.
- Python Software Foundation. Requests: Http for humans, 2024c. URL <https://requests.readthedocs.io/en/latest/user/quickstart/>. Accessed: 2026.
- ridvanaltun. Guacamole rest api authentication, 2024a. URL <https://github.com/ridvanaltun/guacamole-rest-api-documentation/blob/master/docs/AUTHENTICATION.md>. Accessed: 2026.
- ridvanaltun. Guacamole rest api connections, 2024b. URL <https://github.com/ridvanaltun/guacamole-rest-api-documentation/blob/master/docs/CONNECTIONS.md>. Accessed: 2026.
- Abdullah Saniy Roslan. Classroom cybersecurity lab platform: A self-hosted private cloud approach for practical ict education. *Tropical Scientific Journal*, 5(2):112–120, 2026. URL <https://scientificacademic.com/index.php/tsj/article/view/68>.
- Amazon Web Services. Aws academy learner lab, 2024a. URL <https://awsacademy.instructure.com/courses/155047>. Accessed: 2026.
- Amazon Web Services. What is aws?, 2024b. URL <https://aws.amazon.com/what-is-aws/>. Accessed: 2026.
- Amazon Web Services. Aws sdk for python (boto3), 2024c. URL <https://aws.amazon.com/sdk-for-python/>. Accessed: 2026.
- Teamcamp. Burndown chart generator, 2024. URL <https://www.teamcamp.app/resources/burndown-chart-generator>. Accessed: 2026.
- Rebeka Tóth and László Erdődi. Expanding horizons: The evolving landscape of development opportunities in cybersecurity training platforms. In *Norsk IKT-konferanse for forskning og utdanning*. NTNU, 2023. URL <https://www.ntnu.no/ojs/index.php/nikt/article/view/5643>.

A Design of Website Pages (Figma & Website)

The following appendix contains visual references supporting the design and implementation discussed in this report. The design was created in Figma to provide a guideline for what the website pages would look like.

A.1 Choose Role

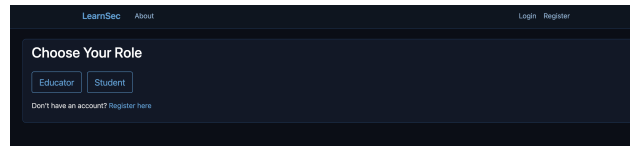


Figure A.1: Choose Role website page

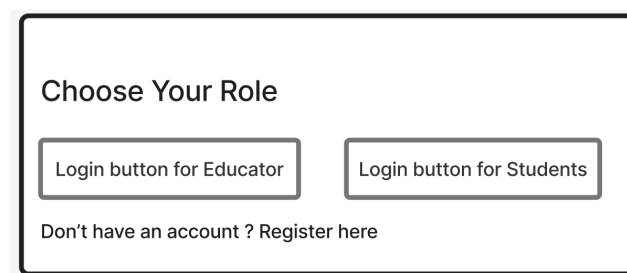


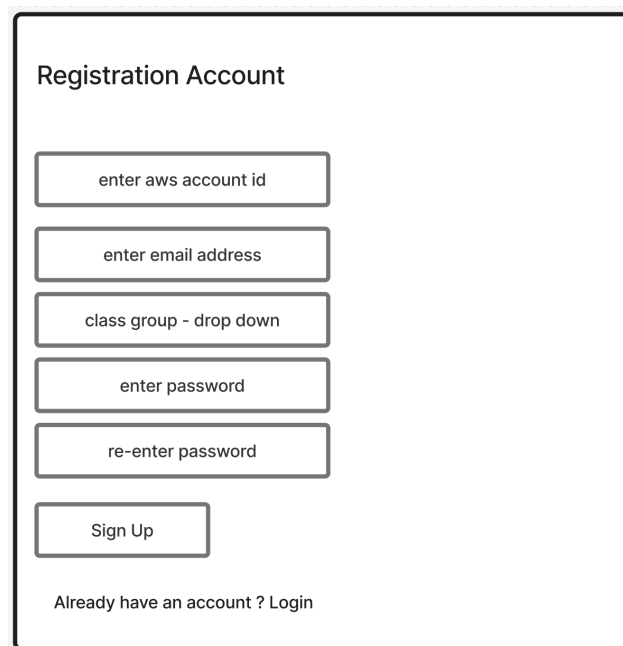
Figure A.2: Choose Role Figma design

The choose role page shows two separate login routes one for educator and one for students. The reason for this design choice is due to the different content and functionality that is accessible to the two types of users. AWS Cognito ensures only the users that are registered and confirmed email addresses, which are assigned to class groups and roles are able to login to their respective role. As this is the first page a user sees when they visit the platform a link to the registration page is included for new users who have not yet created an account.

A.2 Register



Figure A.3: Register website page



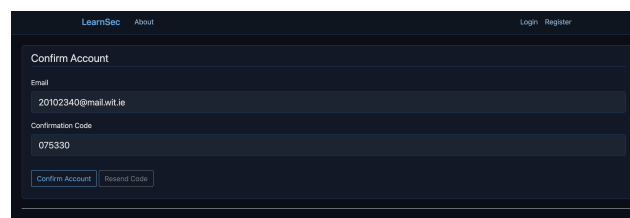
The Figma design shows a registration form titled "Registration Account". It contains the following elements from top to bottom: a text input field labeled "enter aws account id", a text input field labeled "enter email address", a dropdown menu labeled "class group - drop down", a text input field labeled "enter password", a text input field labeled "re-enter password", a "Sign Up" button, and a link that says "Already have an account ? Login".

Figure A.4: Register Figma design

Each field is required for the registration process. They are required for the following reasons:

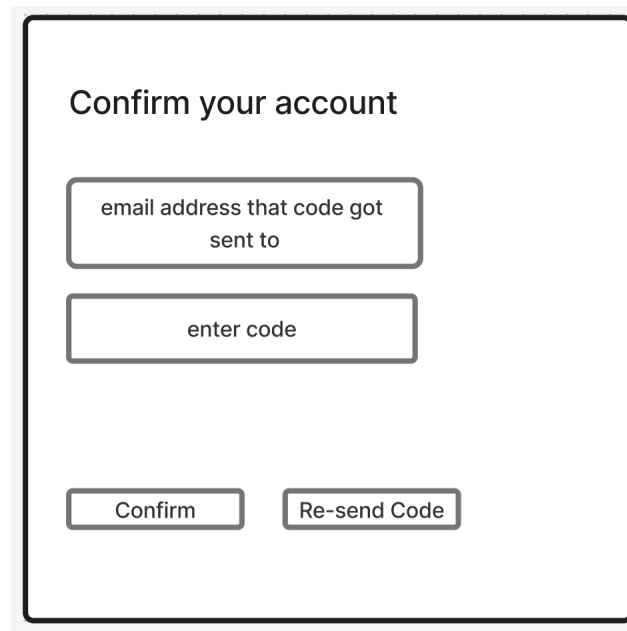
- **AWS Account ID** : collected to identify and associate the user with their AWS account within the platform.
- **Email Address** : used to automatically determine the user's role based on the email prefix. The user does not need to manually select their role during registration (Student is numerical & Educator is alphabetical). The email address is also used by AWS Cognito to send a confirmation code to verify the account. The only emails allowed to be used are restricted to institution email addresses.
- **Class Group** : collected to filter the content visible to the student, ensuring they only see content relevant to their module.
- **Password**: required to secure the user's account.
- **Confirm Password** : ensures the password was entered correctly before the account is created.

A.3 Confirm Account



The screenshot shows a web page titled "Confirm Account". At the top, there are links for "LearnSec", "About", "Login", and "Register". The main content area has two input fields: "Email" with the value "20102340@mail.wit.ie" and "Confirmation Code" with the value "076330". Below these fields are two buttons: "Confirm Account" and "Resend Code". At the bottom, there is a link that says "Already have an account? Log In".

Figure A.5: Confirm Account website page



Confirm your account

email address that code got sent to

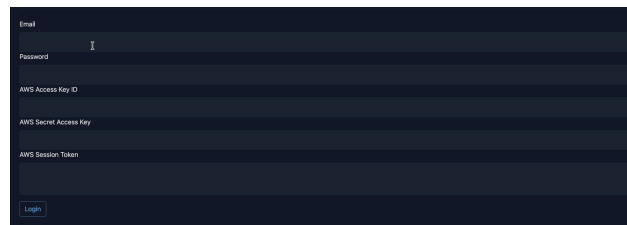
enter code

Confirm Re-send Code

Figure A.6: Confirm Account Figma design

Once a user registers a confirmation code is sent to them through the email they used to register. In AWS Cognito to change the status of the account from "unconfirmed" to "confirmed" the users must enter in the code sent to their email address. There is an option to resend the code in case the user did not receive the email.

A.4 Login Page



Email

Password

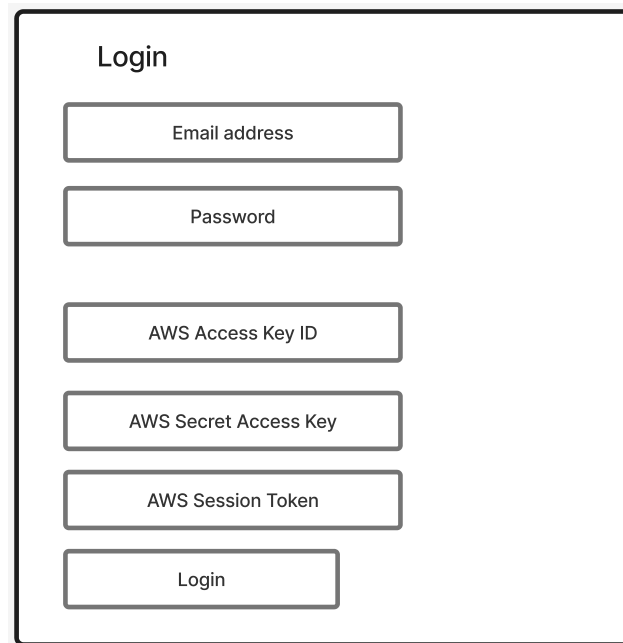
AWS Access Key ID

AWS Secret Access Key

AWS Session Token

Login

Figure A.7: Login Page website page : same for Educator and Student



The image shows a Figma design for a login page. It is a rectangular box with a dark border. Inside, the word "Login" is at the top. Below it are six input fields, each with a placeholder text: "Email address", "Password", "AWS Access Key ID", "AWS Secret Access Key", "AWS Session Token", and "Login". The fields are arranged vertically and are separated by small gaps.

Figure A.8: Login Page figma design : same for Educator and Student

Each field is required for the login process. They are required for the following reasons.

- **Email Address** : used to automatically determine the user's role based on the email prefix and identify they are registered. The user needs to manually select their role during login.
- **Password**: required to secure the user's account.
- **AWS Access Key ID** : a unique identifier for the AWS session
- **AWS Secret Access Key** : a secret key used alongside the Access Key ID to authenticate requests to AWS services
- **AWS Session Token** : a temporary token generated for the session that expires after four hours.

A.5 Account

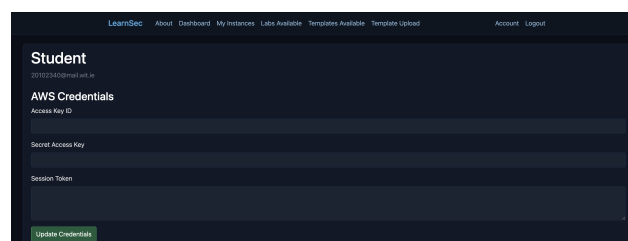


Figure A.9: Account website page

The Figma design for the Account Page includes the following elements:

- Role**: A label above the Email Address field.
- Email Address**: A text input field.
- AWS Credentials**: A section header for the AWS-related fields.
- Access Key ID**: A text input field.
- Secret Access Key**: A text input field.
- Session Token**: A text input field.
- Update Credentials**: A button at the bottom of the form.

Figure A.10: Account Page Figma design

The purpose of the account page is for the user to be able to update their AWS Academy credentials through the website without having to log out and log back in and continue working on their lab environments. These credentials expire every four hours when the AWS Academy learner lab session ends.

AWS Academy provides three temporary credentials that are required to interact with AWS services:

- **AWS Access Key ID** : a unique identifier for the AWS session
- **AWS Secret Access Key** : a secret key used alongside the Access Key ID to authenticate requests to AWS services
- **AWS Session Token** : a temporary token generated for the session that expires after four hours

A.6 Dashboard

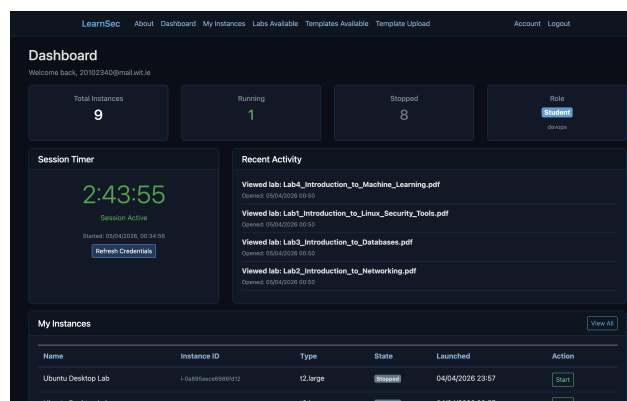


Figure A.11: Dashboard website page

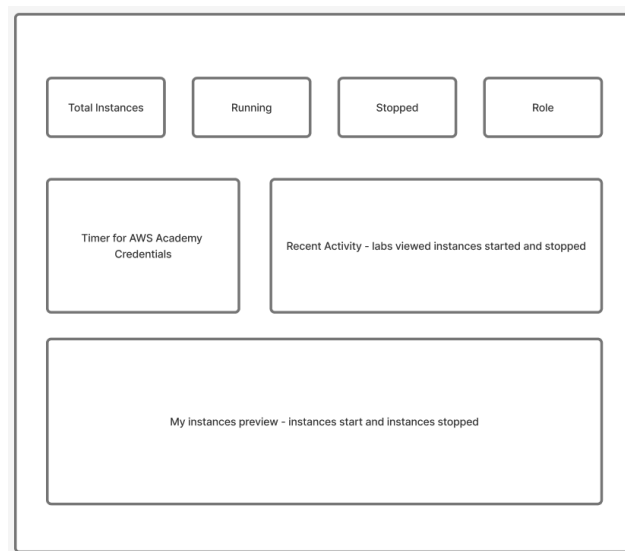


Figure A.12: Dashboard Figma design

The dashboard was designed to give the user a complete overview of their current session and AWS resources in one place as soon as they log in. Instead of the user navigating through each page the user can see this information through a series of cards.

The dashboard cards shows the following to the user:

- **Total instances** : How many instances there are overall.
- **Total Running instances** : How many instances are started.
- **Total Stopped instances** : How many instances are stopped.
- **Timer for AWS Academy Credentials (4 hours starting from login)** : Timer to remind users to update their credentials.
- **Recent activity** : What lab material have been accessed recently.
- **My instances preview** : Preview of the my instances page.

A.7 My Instances

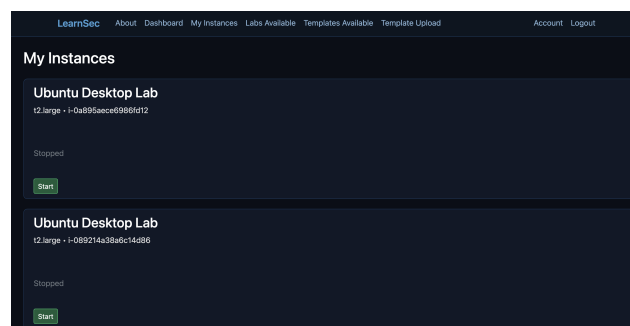


Figure A.13: My instances website page

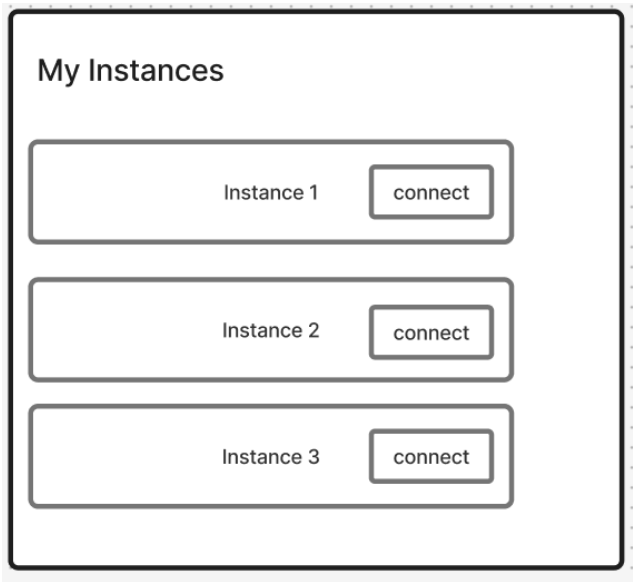


Figure A.14: My Instances Figma design

The design choice for the instances page was to give the user full control and access to their EC2 instances on one page. This page allows for the users to start and stop instances and also see the state that the instance is in, which can be started, stopped or pending. The table layout was chosen to present instance information clearly and concisely, with the user able to interact with the EC2 instances based on the current state of each instance.

A.8 Labs Available

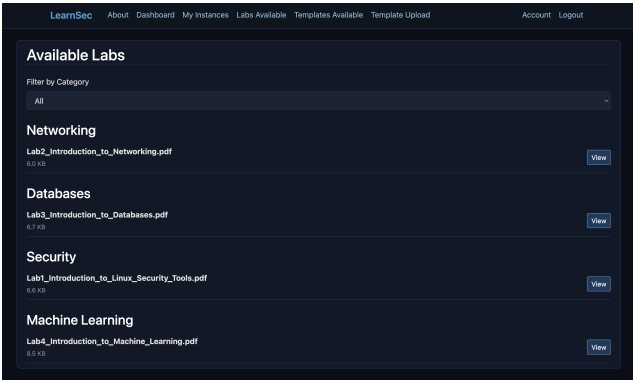


Figure A.15: Labs Available website page for Students

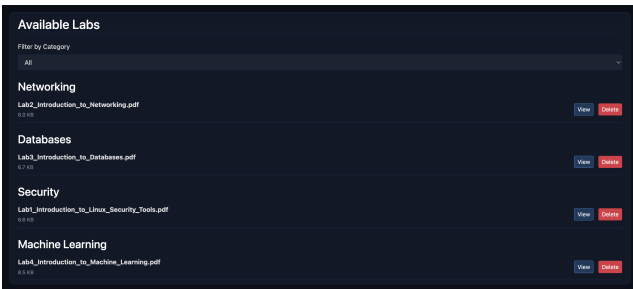


Figure A.16: Labs Available website page for Educators

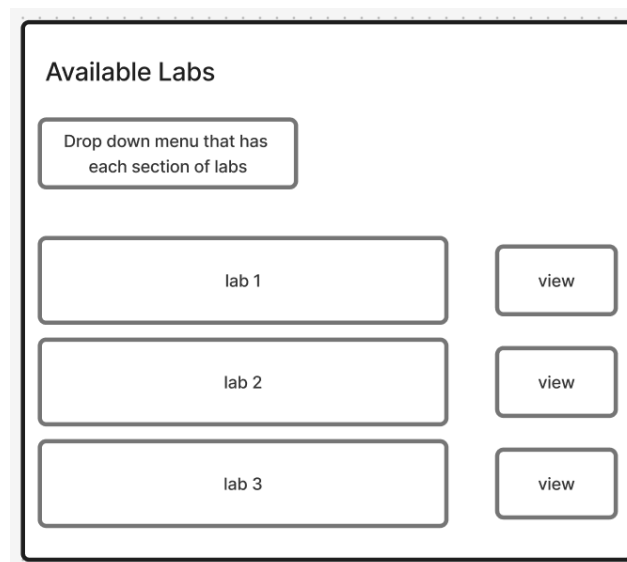


Figure A.17: Lab Available Figma design

The labs available page was designed to show users what lab material is available to them based on their class group. There is a drop down menu to pick specific categories for lab material. Students and educators are able to view the lab material by clicking view and it opens in the browser. Educators have the extra option of being able to remove lab material.

A.9 Labs Upload

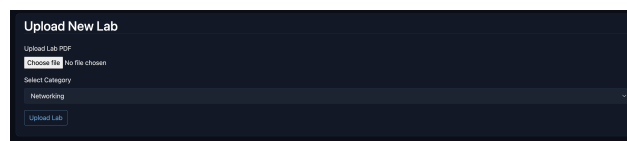


Figure A.18: Lab Upload website page

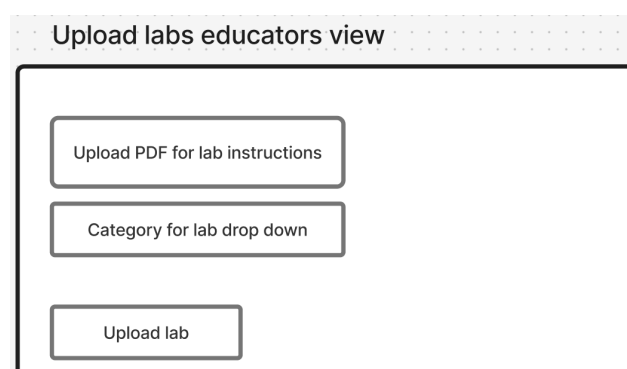


Figure A.19: Lab Upload Figma design

The lab upload page was designed to allow educators to upload lab material and assign them to a category. Educators are able to upload the lab material but students are not allowed to upload they can only view the lab in the browser.

A.10 Templates Available

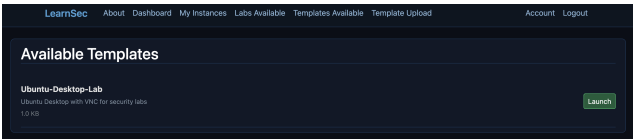


Figure A.20: Templates Available website page for Students

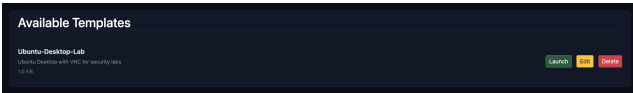


Figure A.21: Templates Available website page for Educators

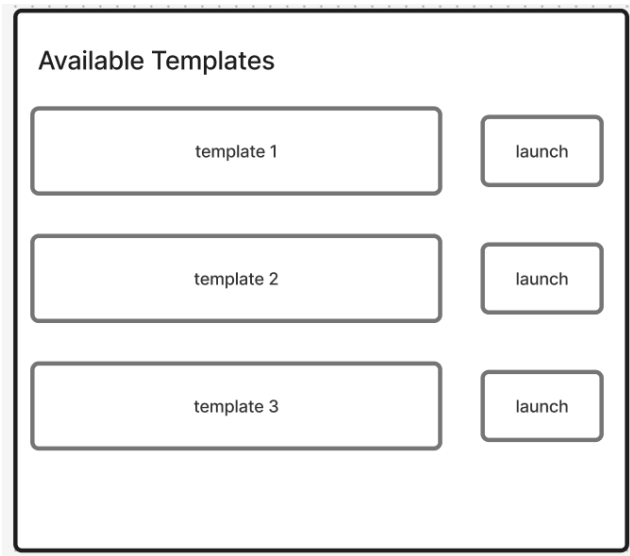


Figure A.22: Templates Available Figma design

The templates available page was designed to show users what launch templates is available to them based on their class group. Students and educators are able to view the launch templates by clicking view and it opens in the browser. Educators have the extra option of being able to remove launch templates.

A.11 Template Upload

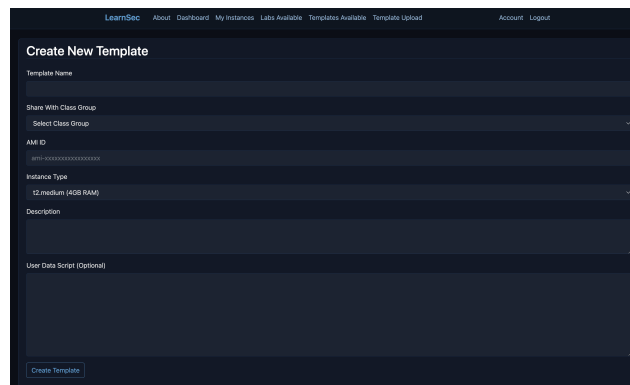
A screenshot of a web application interface for creating a new template. The page has a dark theme. At the top, there is a navigation bar with links: LearnSec, About, Dashboard, My Instances, Lists Available, Templates Available, Template Upload, Account, and Logout. The main heading is 'Create New Template'. Below it, there are several input fields: 'Template Name' (a text input), 'Share With Class Group' (a dropdown menu with 'Select Class Group' as the placeholder), 'AMI ID' (a text input with 'ami-xxxxxxxxxxxxxxxx' as a placeholder), 'Instance Type' (a dropdown menu with 't2.medium (4GB RAM)' as the placeholder), 'Description' (a text area), and 'User Data Script (Optional)' (a text area). At the bottom right, there is a 'Create Template' button.

Figure A.23: Template Upload website page

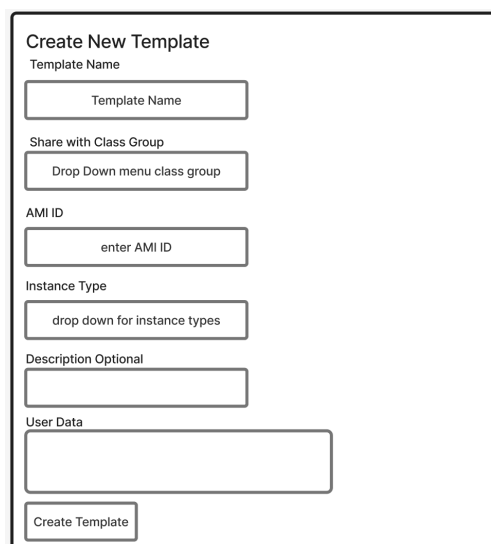
A Figma design of the 'Create New Template' form. The form is titled 'Create New Template' and contains the following fields: 'Template Name' (a text input), 'Share with Class Group' (a dropdown menu with 'Drop Down menu class group' as the placeholder), 'AMI ID' (a text input with 'enter AMI ID' as the placeholder), 'Instance Type' (a dropdown menu with 'drop down for instance types' as the placeholder), 'Description Optional' (a text input), 'User Data' (a text area), and a 'Create Template' button at the bottom.

Figure A.24: Template Upload Figma design

The upload templates page was designed to allow users to create launch templates and assign them to a class group. Students and educators are able to create launch templates. The following are required to create a launch template:

- **Template Name** : required to identify the template on the platform and tag the EC2 instance when it is launched.
- **Share to Class Group** : required to restrict the template to a specific class group ensuring students only see templates relevant to their module.
- **AMI ID** : required to specify which pre-configured VM image to use when launching the instance.
- **Instance Type** : required to define the computing resources allocated to the VM.
- **User Data** : required to provide a script that runs automatically when the instance launches.
- **Description** : an optional field allowing additional context about the template to be provided.

B Sprint Burndown Charts

B.1 Reading the Burndown Charts

Each burndown chart displays the progress made during a sprint. The X axis represents the dates across the sprint duration. On the Y axis this shows what are known as story points. Story points are a way to show the effort and complexity of a task. Each task is assigned a value of story points before the start of each sprint. Each sprint consists of 50 story points overall. A task that is very simple would be assigned between 1 to 4 points whereas a task that requires more effort would be between 5 to 10. At the end of a sprint all story points for the tasks completed in that sprint are added together. It is important to note that task assigned a specific amount of story points may not always reflect the exact value of completion story points. An example of this could be that task A is worth 5 story points may only have 3 story points logged in for that task on a specific date and the remaining 2 story points logged the next day along with another task. It is hard to complete an entire task all in one day so it needs to be spread across 2 days which means it feeds into another day. This is a normal occurrence in sprint development and does not indicate any issues with planning.

The burndown charts show two lines. The grey line represents the ideal flow of the burndown chart showing the rate that work should ideally be done if tasks were distributed evenly across the dates of the sprint. The black line represents the progress actually made through the tasks. The ideal way the graph should look is the black line and the grey line should be very close together and consistent. There is cases of where there is a very productive day showing the black line to drop meaning a significant amount of story points were completed, but this goes the opposite when the days are not as productive. where it remains flat it indicates a day where no story points were completed.

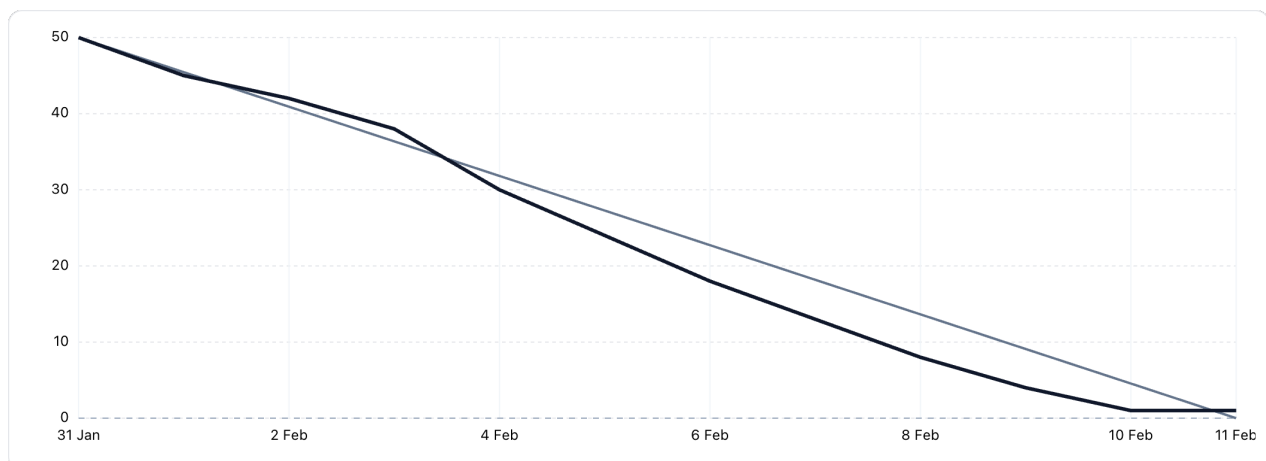


Figure B.1: Sprint 1 Burndown Chart

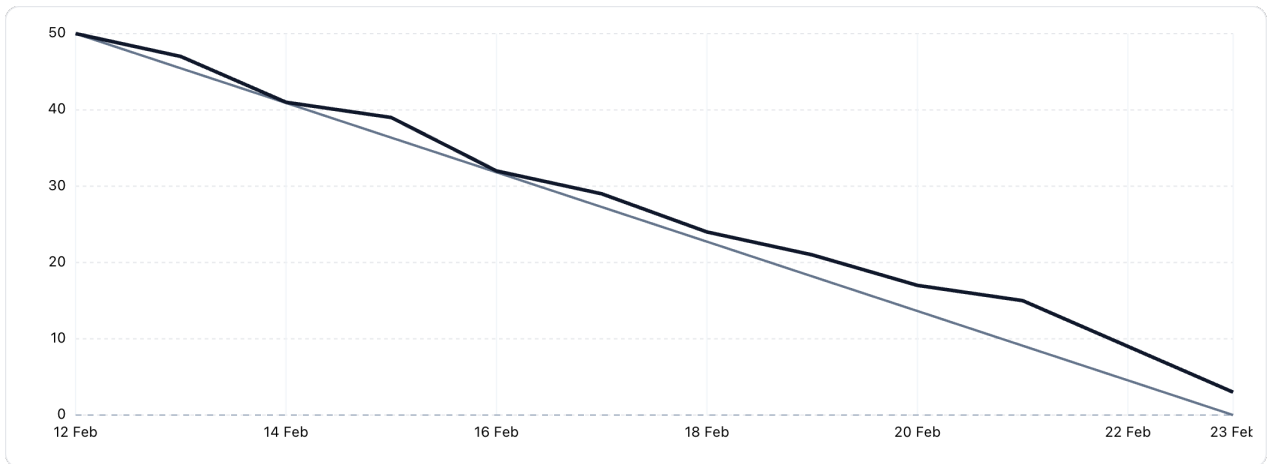


Figure B.2: Sprint 2 Burndown Chart

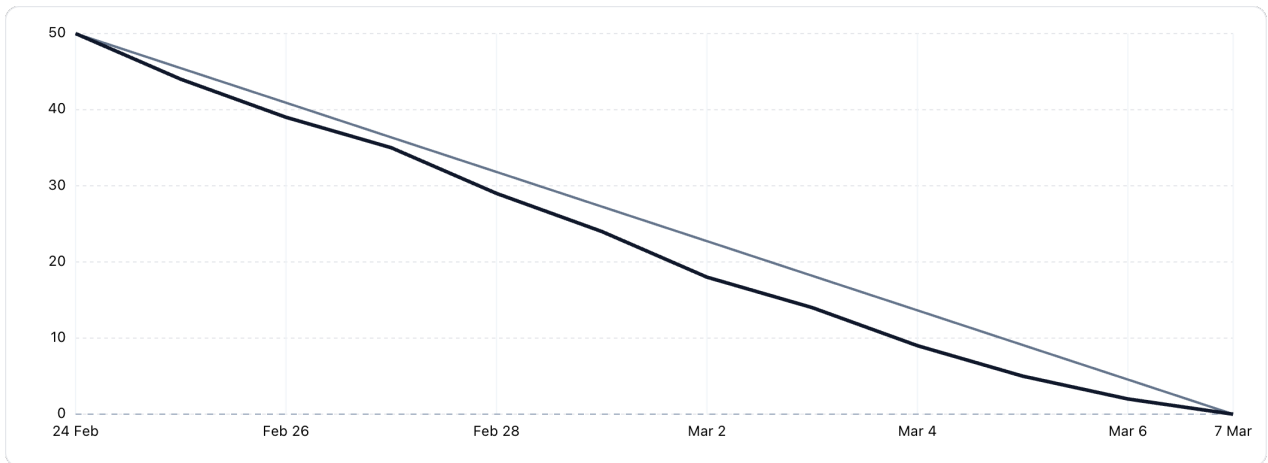


Figure B.3: Sprint 3 Burndown Chart

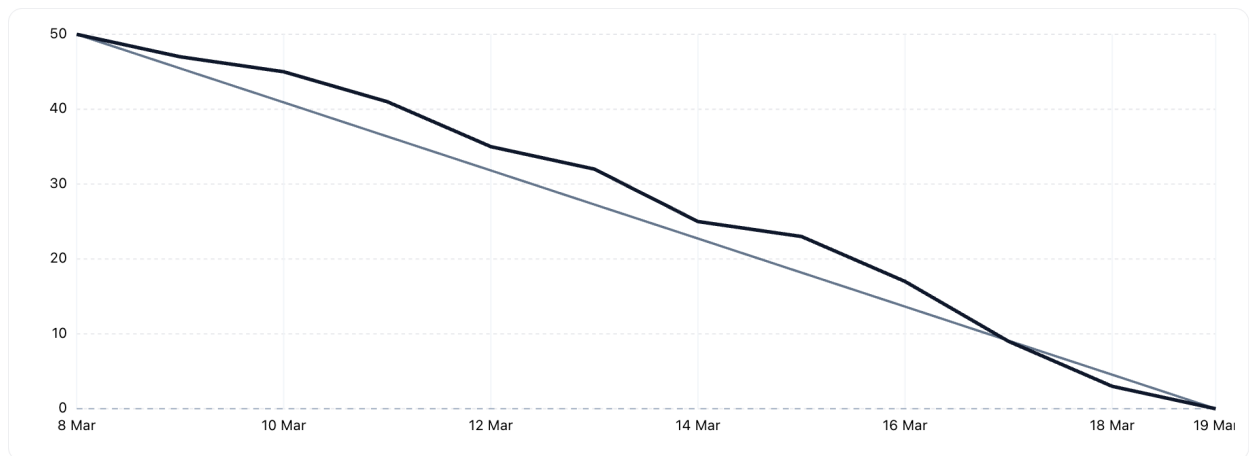


Figure B.4: Sprint 4 Burndown Chart

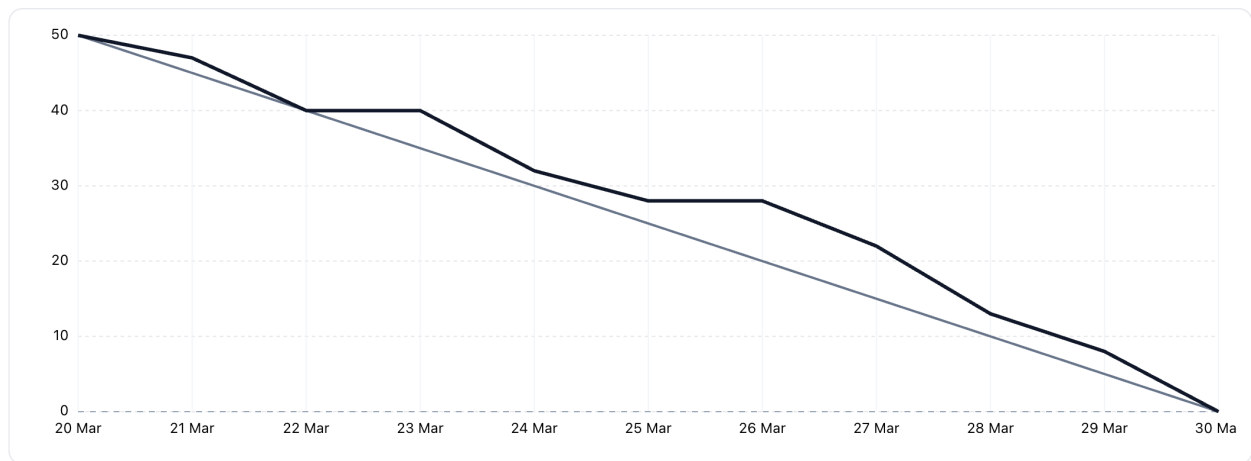


Figure B.5: Sprint 5 Burndown Chart